

Projeto Integrador III

Aula 0 — O básico para acompanhar o curso

INTRODUÇÃO AO R • ENTREGA PARA CASA

Danilo Prado de Lima Silva
Victoria Cabral Quinterio

Motor de Busca • Recuperação de Informação

1 Parte 1 — Explicar, Explorar e Prever

Para cada bloco de código da aula, seguimos os três passos pedidos: primeiro explicamos o que cada linha faz e por que ela existe; depois exploramos o código alterando algum valor, argumento ou nome e observamos o que mudou; por fim, registramos o que esperávamos antes de rodar a versão alterada, comparando com o resultado obtido.

Bloco 1. Atribuição e vetores

```
v <- c(10, 20, 30, 40) # c() = "combine": cria um vetor
v
sum(v)
```

SAÍDA NO CONSOLE

```
[1] 10 20 30 40
[1] 100
```

EXPLICAR

A linha 1 usa o operador `<-` para criar o vetor `v`, guardando quatro números na mesma "caixa"; `c()` existe porque em R quase tudo é vetor, e `c()` é a forma de combinar vários valores soltos em um só objeto. A linha 2, ao digitar apenas `v`, imprime o vetor no console. A linha 3 chama `sum(v)`, uma função pronta do R que percorre o vetor inteiro e devolve a soma de todos os elementos.

EXPLORAR

Alteramos o código acrescentando um quinto valor (50) ao vetor e chamamos também `mean(v)`, que não havia sido usada antes:

```
v2 <- c(10, 20, 30, 40, 50)
v2
sum(v2)
mean(v2)
```

SAÍDA NO CONSOLE

```
[1] 10 20 30 40 50
[1] 150
[1] 30
```

O vetor passou a ter cinco posições em vez de quatro, a soma subiu de 100 para 150 (exatamente 50 a mais, o valor incluído) e a média apareceu como 30, que é 150 dividido por 5.

PREVER

Antes de rodar, esperávamos que `sum(v2)` desse 150 (100 + 50) e que `mean(v2)` desse 30, já que a média de 10, 20, 30, 40, 50 é a soma dividida pela quantidade de elementos (150 / 5). O resultado bateu exatamente com a previsão, o que confirma que `c()` apenas acrescenta elementos ao vetor sem alterar o comportamento das funções agregadoras.

Bloco 2. Vetores nomeados

```
notas <- c(ana = 8, bruno = 6, carla = 9)
notas
notas["bruno"] # acesso pelo NOME, nao pela posicao
```

SAÍDA NO CONSOLE

```
ana bruno carla
  8     6     9
bruno
  6
```

EXPLICAR

A linha 1 cria um vetor onde cada posição recebe um rótulo (ana, bruno, carla) além do valor numérico; esses nomes ficam "colados" a cada elemento. A linha 2 imprime o vetor mostrando nomes em cima e valores embaixo. A linha 3 usa o nome "bruno" entre colchetes para buscar o valor correspondente, em vez de usar a posição numérica -- é exatamente esse mecanismo que, na Aula 01, permite fazer `docs["d5"]` para recuperar um documento pelo seu identificador.

EXPLORAR

Adicionamos um quarto nome ao vetor e buscamos por ele:

```
notas2 <- c(ana = 8, bruno = 6, carla = 9, danilo = 10)
notas2
notas2["danilo"]
```

SAÍDA NO CONSOLE

```
ana  bruno  carla  danilo
  8     6     9     10
danilo
  10
```

O vetor cresceu para quatro posições e a busca por "danilo" devolveu 10, mantendo o nome como rótulo do resultado.

PREVER

Esperávamos que a busca por notas2["danilo"] devolvesse apenas o valor 10, acompanhado do nome danilo como rótulo -- e não a posição 4, já que o acesso é feito pelo nome. A previsão se confirmou: o R nunca considerou a posição do elemento, apenas o rótulo textual, o que reforça por que vetores nomeados são a base do corpus na Aula 01.

Bloco 3. Trabalhando com texto

```
frase <- "recuperacao de informacao"
toupper(frase)
strsplit(frase, " ") # quebra em pedacos -> devolve uma LISTA
```

SAÍDA NO CONSOLE

```
[1] "RECUPERACAO DE INFORMACAO"
[[1]]
[1] "recuperacao" "de"           "informacao"
```

EXPLICAR

A linha 1 guarda uma frase inteira como uma única string de texto. A linha 2 usa toupper() para converter todos os caracteres em maiúsculas, sem alterar a variável original. A linha 3 usa strsplit() para quebrar a string em pedaços sempre que encontra um espaço (" "); o resultado é uma lista contendo um vetor de palavras, e não um vetor simples -- por isso aparece o [[1]] antes das palavras.

EXPLORAR

Trocamos toupper por tolower e usamos uma frase nova, em maiúsculas:

```
frase2 <- "PROJETO INTEGRADOR TRES"
tolower(frase2)
strsplit(frase2, " ")
```

SAÍDA NO CONSOLE

```
[1] "projeto integrador tres"
[[1]]
[1] "PROJETO"      "INTEGRADOR"   "TRES"
```

tolower() fez o oposto de toupper(), como o slide já indicava, e o strsplit() continuou quebrando por espaço -- mas preservando as maiúsculas originais, porque foi aplicado direto sobre frase2 (antes da conversão para minúsculas).

PREVER

Antes de rodar, esperávamos que `tolower(frase2)` devolvesse "projeto integrador tres" todo em minúsculas, e que `strsplit` continuasse devolvendo três pedaços, um por palavra, já que a frase tem três palavras separadas por espaço -- independentemente de estarem em maiúsculas ou minúsculas. O resultado confirmou a previsão quanto à quantidade de pedaços, mas nos fez notar um detalhe importante: como aplicamos `strsplit` sobre `frase2` (não sobre o resultado do `tolower`), as palavras continuaram em maiúsculas. Isso mostra por que, na Aula 01, a ordem das operações importa: primeiro normalizamos com `tolower()` e só depois tokenizamos.

Bloco 4. unlist: de lista para vetor

```
unlist(strsplit(frase, " "))
```

SAÍDA NO CONSOLE

```
[1] "recuperacao" "de" "informacao"
```

EXPLICAR

`strsplit` sempre devolve uma lista, mesmo quando só há uma frase dentro dela. `unlist()` existe para "achatar" essa lista em um vetor simples de palavras, muito mais fácil de manipular depois (contar, comparar, filtrar). O comentário do slide já adianta que, na Aula 01, essa combinação aparece como `unlist(strsplit(texto, "\s+"))`, trocando o espaço simples por uma expressão regular que reconhece qualquer quantidade de espaços.

EXPLORAR

Testamos exatamente essa versão com regex, usando uma frase com espaços duplicados e triplicados de propósito:

```
frase3 <- "recuperacao  de   informacao"
unlist(strsplit(frase3, "\\s+"))
```

SAÍDA NO CONSOLE

```
[1] "recuperacao" "de" "informacao"
```

Mesmo com espaços extras entre as palavras, o resultado final foi idêntico ao original: três palavras limpas, sem strings vazias no meio.

PREVER

Antes de rodar, esperávamos que, se trocássemos " " (um único espaço) por "\s+" (um ou mais espaços) no strsplit de uma frase com espaços duplicados, o resultado continuasse com apenas três palavras -- sem elementos vazios "" entre elas, que é o problema que aconteceria se usássemos apenas " " nesse caso. A previsão se confirmou, e esse é exatamente o motivo pelo qual a Aula 01 usa a regex "\s+" em vez do espaço simples.

Bloco 5. Criando próprias funções

```
dobro <- function(x) {  
  x * 2 # a ultima expressao e o valor devolvido  
}  
dobro(7)
```

SAÍDA NO CONSOLE

```
[1] 14
```

EXPLICAR

A palavra-chave function(x) declara que estamos criando uma função que recebe um parâmetro chamado x. O corpo da função, entre chaves, contém uma única expressão (x * 2), que em R é automaticamente o valor devolvido -- não é preciso escrever return() explicitamente. Ao chamar dobro(7), o R substitui x por 7 e devolve 14. Funções como essa deixam o código reutilizável, o que será essencial para escrever tokenizar() e busca_booleana() mais adiante no curso.

EXPLORAR

Criamos uma segunda função, triplo, seguindo a mesma estrutura:

```
triplo <- function(x) {  
  x * 3  
}  
triplo(7)
```

SAÍDA NO CONSOLE

```
[1] 21
```

Trocando apenas o multiplicador de 2 para 3, o resultado passou de 14 para 21, confirmando que a função é só um "molde" que aplicamos a qualquer valor de entrada.

PREVER

Antes de rodar, esperávamos que `triplo(7)` devolvesse 21, já que $7 * 3 = 21$, seguindo a mesma lógica de `dobro(7) = 14`. O resultado confirmou a previsão. Esse exercício deixa claro que o nome da função e o nome do parâmetro (x) são arbitrários -- o que importa é a operação escrita no corpo da função.

Bloco 6. Aplicando uma função a vários elementos — `lapply`

```
palavras <- list(a = c("x","y","z"), b = c("p","q"))
lapply(palavras, length)
```

SAÍDA NO CONSOLE

```
$a
[1] 3

$b
[1] 2
```

EXPLICAR

A linha 1 cria uma lista nomeada com dois elementos, `a` e `b`, cada um sendo um vetor de letras diferente. A linha 2 usa `lapply()` (list apply) para aplicar a função `length()` a cada elemento da lista, sem precisar escrever um laço `for`. O resultado é sempre outra lista, com um elemento de saída para cada elemento de entrada -- por isso aparecem os rótulos `$a` e `$b`.

EXPLORAR

Acrescentamos um terceiro elemento `c` à lista, com quatro letras:

```
palavras2 <- list(a = c("x","y","z"), b = c("p","q"), c = c("m","n","o","p"))
lapply(palavras2, length)
```

SAÍDA NO CONSOLE

```
$a
[1] 3

$b
[1] 2

$c
[1] 4
```

A lista de saída ganhou um terceiro elemento, `$c`, com o valor 4, mantendo a mesma estrutura -- um resultado por elemento de entrada.

PREVER

Antes de rodar, esperávamos que a saída tivesse três elementos (\$a, \$b, \$c) e que o valor de \$c fosse 4, pois o vetor `c("m","n","o","p")` tem quatro letras. A previsão se confirmou. Isso mostra que `lapply` escala automaticamente para listas de qualquer tamanho, sem precisar reescrever código -- exatamente o que permite tokenizar um corpus inteiro de documentos com uma única chamada.

Bloco 7. `sapply`: o mesmo, mas simplificado

```
sapply(palavras, length)
```

SAÍDA NO CONSOLE

```
a b  
3 2
```

EXPLICAR

`sapply()` faz exatamente o mesmo trabalho que `lapply()` -- aplica `length()` a cada elemento da lista `palavras` --, mas tenta simplificar o resultado para um vetor nomeado (ou matriz), em vez de devolver uma lista. Como todos os resultados aqui são números únicos, o R consegue juntar tudo em um vetor com nomes `a` e `b`, mais compacto de ler que o formato de lista do `lapply`.

EXPLORAR

Testamos `sapply` sobre a lista maior (`palavras2`, com três elementos) e, na sequência, trocamos `sapply` por `lapply` sobre a lista original, para comparar os dois formatos de saída lado a lado:

```
sapply(palavras2, length)  
lapply(palavras, length)
```

SAÍDA NO CONSOLE

```
a b c  
3 2 4  
$a  
[1] 3  
  
$b  
[1] 2
```

O `sapply` sobre `palavras2` devolveu um vetor nomeado com três posições (`a b c`); já o `lapply` sobre `palavras` devolveu uma lista, com o mesmo conteúdo do Bloco 6, mas em formato `$a / $b` em vez de vetor.

PREVER

Esperávamos que `sapply(palavras2, length)` devolvesse um vetor simples com três números nomeados (a, b, c), já que cada resultado é um número único e o `sapply` sempre tenta simplificar nesse caso; e que trocar `sapply` por `lapply` devolveria o mesmo conteúdo numérico, porém embrulhado em formato de lista em vez de vetor. Ambas as previsões se confirmaram, o que resume a regra prática do próprio slide: `lapply` quando se quer lista, `sapply` quando se quer vetor ou matriz.

Bloco 8. Contando com `table`

```
tokens <- c("de", "casa", "de", "rua")
table(tokens) # conta as ocorrências de cada valor
```

SAÍDA NO CONSOLE

```
tokens
casa  de  rua
   1   2   1
```

EXPLICAR

A linha 1 cria um vetor de quatro palavras (com repetição). A linha 2 usa `table()` para contar quantas vezes cada valor único aparece no vetor: "de" aparece duas vezes, "casa" e "rua" aparecem uma vez cada. O resultado já vem ordenado alfabeticamente pelos próprios valores. Esse é o mecanismo básico por trás da frequência de termos em um motor de busca.

EXPLORAR

Acrescentamos mais duas ocorrências de "de" e uma de "praia" ao vetor de tokens:

```
tokens2 <- c("de", "casa", "de", "rua", "de", "praia")
table(tokens2)
```

SAÍDA NO CONSOLE

```
tokens2
casa  de  praia  rua
   1   3   1   1
```

A contagem de "de" subiu de 2 para 3, e um novo termo, "praia", passou a aparecer na tabela com contagem 1 -- algo que não acontecia antes porque "praia" simplesmente não estava no vetor original.

PREVER

Antes de rodar, esperávamos que "de" aparecesse com contagem 3 (já que adicionamos mais uma ocorrência às duas já existentes) e que "praia" surgisse como uma nova linha na tabela, com contagem 1. O resultado confirmou exatamente essa previsão. Isso mostra uma limitação importante do `table()` puro: ele só lista os termos que de fato apareceram no vetor -- o que motiva o uso de `factor` no bloco seguinte.

Bloco 9. `factor`: fixando as categorias

```
vocab <- c("casa", "de", "rua", "praia")
table(factor(tokens, levels = vocab))
```

SAÍDA NO CONSOLE

casa	de	rua	praia
1	2	1	0

EXPLICAR

A linha 1 cria um vocabulário fixo com quatro termos possíveis. A linha 2 envolve `tokens` em `factor(..., levels = vocab)` antes de contar: isso diz ao R "essas são todas as categorias válidas, mesmo que algumas não apareçam nos dados". Por isso "praia" aparece na tabela com contagem 0, mesmo não estando no vetor `tokens` original. Esse é o truque que garante que todo documento vire um vetor do mesmo tamanho (o tamanho do vocabulário), condição necessária para montar a matriz termo-documento.

EXPLORAR

Removemos o factor e comparamos com a contagem simples; depois testamos o comportamento quando tokens contém um termo que não está no vocabulário ("montanha"):

```
table(tokens) # sem factor
tokens3 <- c("de", "casa", "de", "rua", "montanha")
table(factor(tokens3, levels = vocab))
```

SAÍDA NO CONSOLE

```
tokens
casa  de  rua
  1    2   1
casa  de  rua praia
  1    2   1    0
```

Sem o factor, "praia" some da tabela (volta ao comportamento do Bloco 8). Com o factor, mesmo incluindo "montanha" -- um termo fora do vocabulário -- a tabela continua mostrando exatamente as quatro categorias definidas em vocab, com as mesmas contagens de antes; a palavra "montanha" é silenciosamente descartada (vira NA) por não estar entre os levels.

PREVER

Esperávamos que remover o factor fizesse "praia" desaparecer da tabela (voltando ao formato do Bloco 8), já que sem levels fixos o R só mostra o que encontrou. Também esperávamos que um termo fora do vocabulário, como "montanha", não criasse uma quinta categoria na tabela -- já que levels = vocab limita as categorias possíveis a exatamente essas quatro. Ambas as previsões se confirmaram. Isso importa muito para o motor de busca: sem factor, cada documento geraria um vetor de contagem de tamanho diferente (dependendo de quais palavras ele contém), tornando impossível comparar ou multiplicar vetores de documentos diferentes na matriz termo-documento.

Bloco 10. Matrizes

```
m <- matrix(1:4, nrow = 2)
rownames(m) <- c("lin1", "lin2")
colnames(m) <- c("c1", "c2")
m
```

SAÍDA NO CONSOLE

```
      c1 c2
lin1  1  3
lin2  2  4
```

EXPLICAR

A linha 1 cria uma matriz de 2 linhas a partir da sequência 1:4; o R preenche a matriz por coluna, e não por linha -- por isso o resultado é 1,2 na primeira coluna e 3,4 na segunda, e não 1,2,3,4 lidos em linha. As linhas 2 e 3 batizam as linhas e colunas com nomes, o que permite acessar os dados depois por rótulo (`m["lin1", ]`, `m[, "c2"]`, `m["lin1","c2"]`), do mesmo jeito que fizemos com os vetores nomeados no Bloco 2.

EXPLORAR

Criamos uma matriz maior, de 3 linhas por 3 colunas, a partir da sequência 1:9:

```
m2 <- matrix(1:9, nrow = 3)
rownames(m2) <- c("lin1", "lin2", "lin3")
colnames(m2) <- c("c1", "c2", "c3")
m2
```

SAÍDA NO CONSOLE

```
      c1 c2 c3
lin1   1  4  7
lin2   2  5  8
lin3   3  6  9
```

O padrão de preenchimento por coluna se manteve: a primeira coluna recebeu 1,2,3, a segunda 4,5,6 e a terceira 7,8,9 -- confirmando que o R sempre "desce" pelas linhas de uma coluna antes de passar para a coluna seguinte.

PREVER

Antes de rodar, esperávamos que a matriz `m2` preenchesse a primeira coluna com 1, 2, 3, a segunda com 4, 5, 6 e a third with 7, 8, 9, seguindo a mesma lógica "por coluna" observada na matriz `m` original (onde 1,2 formou a primeira coluna). O resultado confirmou a previsão. Entender essa ordem de preenchimento é essencial para não errar a montagem da matriz termo-documento na Aula 01, onde cada coluna deve corresponder a um documento específico.

Bloco 11. Operações úteis em matrizes

```
peso <- c(2, 3) # um peso por LINHA
m * peso # reciclagem: multiplica linha a linha
```

SAÍDA NO CONSOLE

```
      c1 c2
lin1   2  6
lin2   6 12
```

EXPLICAR

O trecho principal cria um vetor peso com um peso para cada linha da matriz m e multiplica $m * \text{peso}$. O R usa reciclagem: como peso tem 2 elementos e m tem 2 linhas, cada peso é aplicado à linha correspondente, repetindo-se ao longo das colunas -- por isso lin1 (valores 1 e 3) foi multiplicada por 2, virando 2 e 6, e lin2 (valores 2 e 4) foi multiplicada por 3, virando 6 e 12.

EXPLORAR

Testamos peso com 3 elementos em vez de 2, sobre a mesma matriz m (que tem apenas 2 linhas):

```
peso3 <- c(2, 3, 4)
m * peso3
```

SAÍDA NO CONSOLE

```
AVISO: longer object length is not a multiple
of shorter object length
      c1 c2
lin1  2 12
lin2  6  8
```

O R não travou, mas emitiu um aviso (warning) porque o comprimento de peso3 (3) não é múltiplo do número de linhas de m (2). Mesmo assim, ele continuou reciclando os pesos na ordem 2, 3, 4, 2, 3, 4, ... percorrendo a matriz por coluna, o que produziu um resultado diferente do esperado e, na prática, sem sentido para o nosso caso de uso.

PREVER

Antes de rodar, esperávamos que a reciclagem "quebrasse" de alguma forma, já que 3 pesos não dividem exatamente 2 linhas -- mas não tínhamos certeza se o R geraria um erro ou apenas um aviso. O resultado mostrou que o R não interrompe a execução: ele apenas avisa e segue reciclando os pesos na ordem em que a matriz é lida internamente (por coluna), o que embaralha completamente a correspondência "um peso por linha" que havíamos planejado. Essa é uma lição importante para o `tfidf <- tdm * idf` da Aula 01.

Bloco 12. Comparações e o operador %in%

```
c("casa", "aviao") %in% vocab # cada elemento esta em vocab?
```

SAÍDA NO CONSOLE

```
[1] TRUE FALSE
```

EXPLICAR

O operador `%in%` testa, elemento a elemento, se cada valor do vetor à esquerda existe dentro do vetor à direita, devolvendo um vetor lógico do mesmo tamanho. Aqui, "casa" está em vocab (TRUE), mas "aviao" não está (FALSE). Combinado com a negação `!`, esse operador serve para remover stopwords: `t[!t %in% stopwords]` mantém apenas os termos de `t` que não estão na lista de stopwords.

EXPLORAR

Testamos o `%in%` com um vetor de teste diferente e também aplicamos a remoção de stopwords e a ordenação decrescente mencionadas no slide:

```
c("praia","trem","casa") %in% vocab
t <- c("casa","de","aviao","rua")
stopwords <- c("de")
t[!t %in% stopwords]
sort(c(3,1,4,1,5), decreasing = TRUE)
```

SAÍDA NO CONSOLE

```
[1] TRUE FALSE TRUE
[1] "casa" "aviao" "rua"
[1] 5 4 3 1 1
```

"praia" e "casa" deram TRUE (estão em vocab) e "trem" deu FALSE; ao remover a stopwords "de", o vetor `t` ficou apenas com "casa", "aviao" e "rua"; e a ordenação decrescente colocou o 5 primeiro e os dois 1 por último.

PREVER

Esperávamos que "praia" e "casa" retornassem TRUE (ambas estão no vetor vocab definido no Bloco 9) e que "trem" retornasse FALSE, por não pertencer a esse vocabulário. A previsão se confirmou exatamente. Já para a remoção de stopwords, esperávamos que apenas "de" fosse eliminado do vetor `t`, restando as outras três palavras na mesma ordem original -- o que também se confirmou, mostrando como `%in%` junto com `!` é a base da limpeza de texto.

Bloco 13. Regex na prática

```
palavras13 <- c("casa", "cachorro", "praia", "cidade")
grep("^ca", palavras13, value = TRUE) # comecam com "ca"?
grepl("a$", palavras13)              # terminam em "a"?
sub("a", "@", "banana")               # troca a PRIMEIRA ocorrencia
gsub("a", "@", "banana")              # troca TODAS
```

SAÍDA NO CONSOLE

```
[1] "casa"      "cachorro"
[1] TRUE FALSE  TRUE FALSE
[1] "b@nana"
[1] "b@n@n@"
```

EXPLICAR

grep(..., value = TRUE) devolve os próprios elementos do vetor que combinam com o padrão -- aqui, os que começam com "ca" (^ca), ou seja, "casa" e "cachorro". grepl() devolve um vetor lógico (TRUE/FALSE) indicando quais elementos terminam em "a" (a\$). sub() substitui apenas a primeira ocorrência do padrão por outro texto; gsub() ("g" de global) substitui todas as ocorrências -- por isso sub trocou só o primeiro "a" de "banana", e gsub trocou os três.

EXPLORAR

Trocamos os padrões: procuramos palavras que terminam em "a" em vez de começar com "ca", testamos quais começam com "ci", e trocamos a letra "c" (em vez de "a") em "cachorro":

```
grep("a$", palavras13, value = TRUE)
grepl("^ci", palavras13)
sub("c", "K", "cachorro")
gsub("c", "K", "cachorro")
```

SAÍDA NO CONSOLE

```
[1] "casa" "praia"
[1] FALSE FALSE FALSE  TRUE
[1] "Kachorro"
[1] "KaKhorro"
```

Com "a\$", apenas "casa" e "praia" combinaram (terminam em "a"); com "^ci", só "cidade" deu TRUE. Em "cachorro", o sub trocou apenas o primeiro "c" (virando "Kachorro"), enquanto o gsub trocou os dois "c" da palavra (virando "KaKhorro").

PREVER

Antes de rodar, esperávamos que "a\$" selecionasse as palavras que terminam em "a" -- "casa" e "praia" -- e que "^ci" desse TRUE somente para "cidade", a única que começa com essas duas letras. Também esperávamos que sub("c","K",...) trocasse só o primeiro "c" de "cachorro" (que tem dois) e que gsub trocasse ambos. Todas as previsões se confirmaram, reforçando a regra prática do próprio slide: sub troca uma vez, gsub troca todas.

2 Parte 2 — Perguntas para investigar

1. O que acontece se você trocar `sapply` por `lapply` no exemplo? E o contrário?

```
lapply(palavras, length) # no lugar de sapply
sapply(palavras, length) # o padrao do slide
```

SAÍDA NO CONSOLE

```
$a
[1] 3

$b
[1] 2

a b
3 2
```

Ao usar `lapply` no lugar de `sapply` na montagem da matriz termo-documento, a estrutura resultante seria uma lista de vetores, inviabilizando operações matriciais diretas de álgebra linear. Se usarmos `sapply` na tokenização em vez de `lapply`, ele tentará forçar a saída para uma matriz de palavras, gerando erros se os documentos possuírem tamanhos diferentes.

2. Em `table(factor(tokens, levels = vocab))`, o que muda se remover o factor?

```
table(factor(tokens, levels = vocab)) # com factor
table(tokens)                       # sem factor
```

SAÍDA NO CONSOLE

```
casa de rua praia
  1   2   1   0

tokens
casa de rua
  1   2   1
```

Sem o factor, a função `table` retornaria apenas a contagem das palavras existentes. Para um motor de busca, isso inviabiliza a criação da Matriz Termo-Documento, pois cada documento (coluna) teria um tamanho de vetor diferente.

3. Na reciclagem $m * \text{peso}$, o que acontece se peso tiver 3 elementos em vez de 2?

```
peso3 <- c(2, 3, 4)
m * peso3
```

SAÍDA NO CONSOLE

```
AVISO: longer object length is not a multiple
of shorter object length
      c1 c2
lin1  2 12
lin2  6  8
```

PREVER E EXPLORAR

O R não interrompe a execução: ele apenas emite um *warning* avisando que o comprimento do objeto maior não é múltiplo do menor, e segue reciclando os pesos na ordem. Essa é uma armadilha silenciosa para cálculos de TF-IDF.

4. Crie um vetor nomeado com 3 frases suas, tokenize e conte os termos

```
frases <- c(
  f1 = "o porto de santos e o maior da america latina",
  f2 = "cubatao fica na baixada santista perto do porto",
  f3 = "o motor de busca organiza os textos do porto de santos"
)
tokens_list <- lapply(frases, function(x) unlist(strsplit(tolower(x), "\\s+")))
todos_tokens <- unlist(tokens_list)
sort(table(todos_tokens), decreasing = TRUE)
```

SAÍDA NO CONSOLE

```
todos_tokens
      de      o      porto      do      santos america
      3      3      3      2      2      1
baixada  busca cubatao  da      e      fica
      1      1      1      1      1      1
latina   maior   motor   na organiza  os
      1      1      1      1      1      1
perto santista  textos
      1      1      1
```

O lapply aplica a limpeza e tokenização a todas as frases nomeadas. A contagem revela quais temas e stopwords são mais frequentes no corpus simulado.

3 Parte 3 — Exercício final: O faxineiro de manchetes

Vetor Base. Manchetes coletadas

```
manchetes <- c(
  "Porto de Santos bate recorde em julho - A Tribuna",
  "cubatao registra melhora na qualidade do ar - A Tribuna",
  "Guaruja tera nova linha de onibus em 2026 - A Tribuna",
  "Sao Vicente inaugura escola no Parque Bitaru - A Tribuna",
  "Santos e Guaruja discutem travessia de balsa - A Tribuna"
)
manchetes
```

SAÍDA NO CONSOLE

```
[1] "Porto de Santos bate recorde em julho - A Tribuna"
[2] "cubatao registra melhora na qualidade do ar - A Tribuna"
[3] "Guaruja tera nova linha de onibus em 2026 - A Tribuna"
[4] "Sao Vicente inaugura escola no Parque Bitaru - A Tribuna"
[5] "Santos e Guaruja discutem travessia de balsa - A Tribuna"
```

Missão 1. Remover o sufixo " - A Tribuna"

```
limpas <- sub(" - A Tribuna$", "", manchetes)
limpas
```

SAÍDA NO CONSOLE

```
[1] "Porto de Santos bate recorde em julho"
[2] "cubatao registra melhora na qualidade do ar"
[3] "Guaruja tera nova linha de onibus em 2026"
[4] "Sao Vicente inaugura escola no Parque Bitaru"
[5] "Santos e Guaruja discutem travessia de balsa"
```

O \$ é essencial aqui porque ancora o padrão no *final* da string, garantindo que apenas o sufixo que aparece no fim de cada manchete seja removido.

Missão 2. Trocar dois ou mais espaços por um só

```
suja <- "Porto de Santos bate recorde"
gsub("\\s{2,}", " ", suja)
```

SAÍDA NO CONSOLE

```
[1] "Porto de Santos bate recorde"
```

Usamos `gsub`, e não `sub`, porque uma manchete pode ter vários trechos com espaçamento duplicado ou triplicado espalhados pelo texto. O `gsub` substitui todas as ocorrências.

Missão 3. Anos (4 dígitos)

```
grep("[0-9]{4}", limpas, value = TRUE)
```

SAÍDA NO CONSOLE

```
[1] "Guaruja tera nova linha de onibus em 2026"
```

O padrão `[0-9]{4}` procura por exatamente quatro caracteres numéricos consecutivos em qualquer posição do texto.

Missão 4. Municípios (ignorando maiúsculas)

```
grep("Guaruja|Cubatao", limpas, value = TRUE, ignore.case = TRUE)
```

SAÍDA NO CONSOLE

```
[1] "cubatao registra melhora na qualidade do ar"  
[2] "Guaruja tera nova linha de onibus em 2026"  
[3] "Santos e Guaruja discutem travessia de balsa"
```

O operador `|` funciona como um "ou" lógico. O parâmetro `ignore.case = TRUE` é indispensável, pois a manchete de Cubatão começa com letra minúscula.

Missão 5. A pegadinha (Porto vs Cidade)

```
grepl("Santos", limpas)
```

SAÍDA NO CONSOLE

```
[1] TRUE FALSE FALSE FALSE TRUE
```

O uso isolado do Regex identificará tanto a cidade de Santos quanto a Autoridade Portuária ("Porto de Santos"). O Regex realiza validação de padrão (sintaxe), mas carece de inteligência contextual (semântica). Resolver isso exige semântica, como análise por tensores e reconhecimento de entidades.